App Dev Project Report

1. Student Details

Name: Arpit Benjamin
Roll Number: 25F1002206
Email: 25f1002206@ds.study.iitm.ac.in

About Me: I'm a student at IITM BS degree, I'm also pursuing BTech-CSE from Birla Institute of Technology, Mesra.

2. Project Details

Project Title: Hospital Management System (HMS)

Problem Statement

Hospitals often rely on manual labor or outdated software to manage patients, doctors, appointments, and treatment history. This leads to conflicts in timings, difficulty in tracking records and appointments, poor transparency and management, poor healthcare services and time-consuming administrative work.

The goal of this project is to build a web-based Hospital Management System that enables Admins, Doctors, and Patients to interact with the system based on their roles, ensuring efficient and elegant appointment handling, patient history management, and secure access to hospital data.

Approach

The Hospital Management System is currently structured as a Flask backend with an API-first architecture and a Vue 3 single-page frontend mounted through a single Jinja2 entry template.

- The database is managed with SQLAlchemy on SQLite, with models for *Admin, Doctor, Patient, Department, Appointment, DoctorAvailability, Treatment, Medicine, and ExportJob* created programmatically.
- User roles (Admin, Doctor, Patient) have separate dashboard data, permissions, and workflows implemented through session-based authentication.
- The website includes appointment booking, rescheduling and cancellation, doctor availability scheduling, treatment/history management, admin search and blacklist controls, and appointment status tracking (Booked/Completed/Cancelled).
- Redis is used for caching, and Redis + Celery run the background jobs like daily summaries, patient reminders, monthly doctor activity reports, and async patient treatment CSV exports.
- The UI is made using Vue 3, custom JavaScript, HTML, CSS, and a single Jinja2 Single Page Application rather than multiple server-rendered dashboard pages.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) to assist with

- Refining part of SQLAlchemy model structure and generation of sample data for testing.
- Improving naming, wording, and overall code/documentation consistency.
- Detection of redundant code after migrating to Vue for UI to ease the migration process.
- Minor frontend UI wording and style consistency (color, icons etc).

The extent of AI/LLM usage is limited to code suggestions and documentation structure. **All final logic, debugging, database design, implementation, and testing were done manually.**

4. Technologies and Frameworks Used

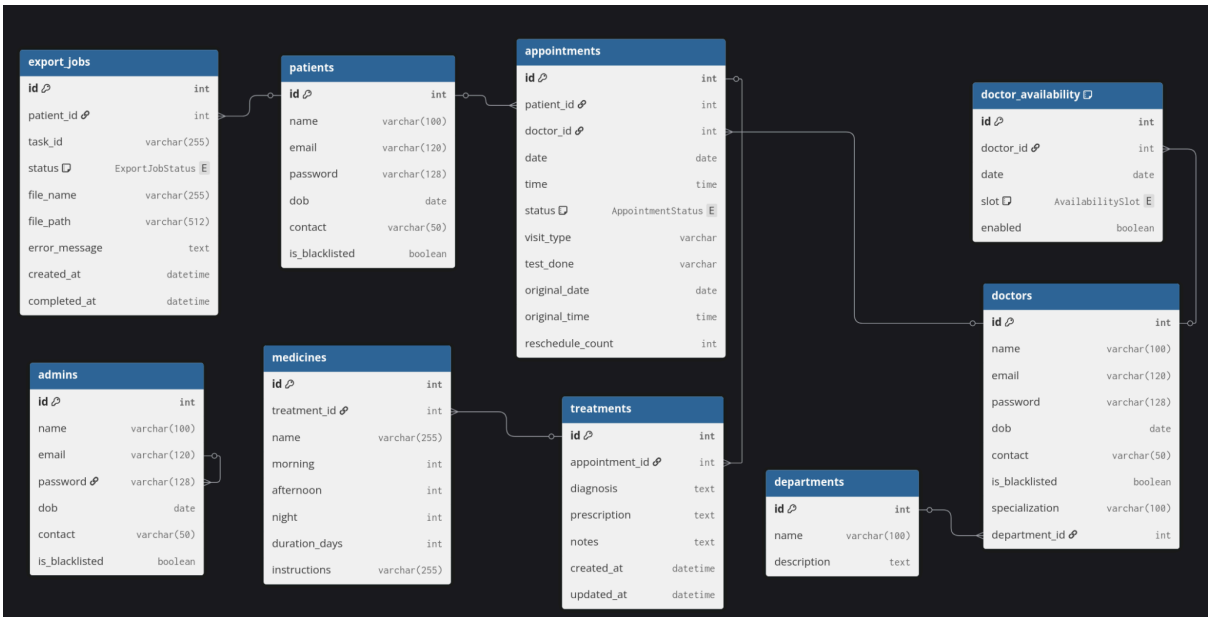
- **Flask:** Core backend framework used for application setup, routing, session handling, and JSON API endpoints.
- **Vue 3:** Frontend framework used to power the single-page application interface in the browser.
- **Jinja2:** Used primarily to serve the single SPA entry template rather than multiple server-rendered dashboard pages.
- **SQLite:** Local relational database used for storing application data.
- **SQLAlchemy:** ORM used to define models, relationships, and create database tables programmatically.
- **Redis:** Used for caching application data and for background processing.
- **Celery:** Used for asynchronous and scheduled jobs such as reminders, reports, and CSV export generation.
- **HTML, CSS, JavaScript:** Used for the application structure, styling, and client-side interactivity, with custom frontend logic in the SPA.

5. Database Schema / ER Diagram

Table	Purpose	Fields
Admin	Stores admin user details	id, name, email, password, dob, contact, is_blacklisted
Department	Hospital departments	id, name, description
Doctor	Doctor details + linked department	id, name, email, password, dob, contact, is_blacklisted, specialization, department_id
Patient	Patient profiles	id, name, email, password, dob, contact, is_blacklisted
Appointment	Records patient–doctor appointments	id, patient_id, doctor_id, date, time, status, visit_type, test_done,

		original_date, original_time, reschedule_count
Treatment	One treatment linked to one appointment	id, appointment_id, diagnosis, prescription, notes, created_at, updated_at
Medicine	Medicines prescribed under a treatment	id, treatment_id, name, morning, afternoon, night, duration_days, instructions
DoctorAvailability	Doctor availability by date & slot	id, doctor_id, date, slot, enabled
ExportJob	Tracks patient treatment CSV export jobs	id, patient_id, task_id, status, file_name, file_path, error_message, created_at, completed_at

Relationships



One-to-Many

- **Department → Doctor** (One department can have many doctors)
- **Doctor → Appointment** (One doctor can have many appointments)
- **Patient → Appointment** (One patient can have many appointments)
- **Treatment → Medicine** (One treatment can have many medicines)
- **Doctor → DoctorAvailability** (One doctor can have many availability entries)
- **Patient → ExportJob** (One patient can have many export jobs)

One-to-One

- **Appointment** → **Treatment** (Each appointment can have one treatment record)

6. API Resource Endpoints

The system exposes JSON API endpoints for login, booking and managing appointments, updating doctor availability, searching records, and admin-level user management. All endpoints are implemented as Flask Blueprints, use SQLAlchemy models for data access, and apply session and role checks through custom decorators.

Blueprints

- `api_bp` – All JSON API endpoints (booking, cancel, reschedule, availability, search, etc.).
- `auth_api_bp` – JSON authentication endpoints for session lookup, login, signup, and logout.
- `auth_login_bp` – Login, logout.

7. Architecture and Features

Architecture

- **app.py** – Main Flask entry point; creates the app, loads configuration, initializes the database setup, registers Blueprints, and serves the SPA entry template.
- **models/** – Contains the SQLAlchemy ORM models for Admin, Department, Doctor, Patient, Appointment, Treatment, Medicine, DoctorAvailability, and ExportJob.
- **routes/** – Route layer organized into Blueprints
- **auth.py** -> authentication API endpoints and routes for login/signup
- **api_routes.py** -> main JSON API for dashboards, profiles, appointments, history, search, blacklist actions, exports, and batch tasks
- **templates/** – Jinja2 layer now centered around a single active SPA entry file (`index.html`) plus embedded component templates used by the frontend.
- **static/** – Frontend assets including the Vue SPA logic in `static/js/app.js`, custom CSS, and static resources.
- **utils/** – Constants, Redis cache helpers, notification/export utilities, database setup, and sample data generation.
- **tasks.py** – Celery background tasks for daily summaries, patient reminders, monthly doctor activity reports, and asynchronous CSV exports.
- **celery_app.py** – Celery application setup, Flask app binding, and scheduled task configuration.
- **config.py** – Centralized application configuration for Flask, SQLite, Redis, Celery, mail, and export settings.

Implemented Features

Admin

- Pre-existing admin account created programmatically during initial database setup
- Add, update, and delete doctor profiles

- Add and update patient profiles
- View all appointments, separated into upcoming and past
- Search doctors, patients, and matching appointments
- Blacklist or unblacklist doctors and patients
- Dashboard showing total doctors, patients, and appointments
- Trigger batch jobs for daily summary, daily reminders, and monthly doctor activity reports
- Track background task status

Doctor

- Login and logout
- View upcoming booked appointments and completed appointments
- View all assigned patients
- Mark owned appointments as Completed or Cancelled
- Add and update visit history, including visit type, tests done, diagnosis, prescription, notes, and medicines
- Provide availability for the next 7 days using morning, afternoon, and evening slots
- View full medical history of assigned patients
- Access patient export records for patients they have treated

Patient

- Register, login, and update profile
- View departments and doctors, including specializations
- View doctor availability for the next 7 days
- Book, reschedule, or cancel appointments
- View upcoming appointments
- Access full medical and treatment history
- Request asynchronous CSV export of treatment history and download completed exports

Other Functionalities

- Prevent double-booking for the same doctor/date/time and for the same patient/date/time
- Enforce availability-slot checks before booking or rescheduling
- Support search by doctor name, specialization, department, email, or contact depending on user role
- Invalidate cached app/dashboard/search data after writes to keep the SPA in sync
- Create database tables entirely from Python SQLAlchemy models and setup code, with no manual schema editing required

8. Video Presentation

Drive Link

(https://drive.google.com/file/d/1_ss_vcjDrNpdV3EDfSmdBrBltQVRhQcm/view?usp=drive_link)